

Introduction

To create a digital enhancement program, contrast adjustment using histogram equalisation on a cumulative intensity histogram was implemented, which back-projects the original image's intensities to create an image of equalised intensities. The following report details how the program works and showcases the enhanced images, as well as the results and run times of the optimisations made.

Implementation

Introduction

This assignment implements a parallelised histogram equalisation algorithm, using OpenCL and C++, which enhances contrast for greyscale and an attempt at colour images. This program processes 8-bit images using GPU acceleration and evaluates performance via kernel execution times.

Using classic parallel patterns such as histogram computation, inclusive scan and LUT mapping, I have tested the program using .pgm and .ppm images, with fixed and variable dimensions.

Histogram equalisation is a technique that adjusts the pixel values of an image based on its intensity histogram (Histogram Equalization - an overview | ScienceDirect Topics, n.d.). This results in an enhanced image visibility by utilising the full dynamic range.

Implementation

To write the code, I used C++ and Visual Studio, along with the CImg library to load and save the greyscale (.pgm) and colour (.ppm) images. To perform the image processing steps in parallel, I used OpenCL, which allows for GPU acceleration. The code consists of platform and device selection (Figure 1), where platform selection finds the OpenCL platform, for example Intel or NVIDIA, and device selection finds the compute device, such as the CPU or GPU. Buffers (Figure 2) are dynamically allocated according to the defined number of histogram bins (num_bins), enabling flexible control of histogram resolution.

The OpenCL kernel code defines five main kernels. `histogram_kernel` computes the image histogram using atomic operations to avoid race conditions. `scan_kernel` performs an inclusive scan using the Hillis-Steele pattern to compute the cumulative histogram. `normalise_kernel` then normalises the cumulative histogram by scaling values using the detected minimum and maximum active bins to create a valid LUT (Lookup tables). Lookup tables are a predetermined array of numbers that provide a shortcut for a specific computation. `apply_lut_kernel` uses this LUT to remap the pixel values of the image to produce the equalised version. For colour images, `apply_lut_kernel_colour` handles the three channels (R, G, B) separately.

Beyond the code used from the workshops, I have also implemented a parameter called `num_bins`, where the histogram resolution is configurable, and the kernels are built to support this dynamically. Full colour support was

added, including the RGB channel separation and combined LUT application. However, I was unable to complete this, as the colour images remained noisy and distorted.

```
// OpenCL setup
vector<cl::Platform> platforms;
cl::Platform::get(&platforms);
cl::Platform platform = platforms.front();

vector<cl::Device> devices;
platform.getDevices(CL_DEVICE_TYPE_GPU, &devices);
cl::Device device = devices.front();
```

Figure 1: Platform and Device Selection

```
cl::Buffer buf_input(context, CL_MEM_READ_ONLY | CL_MEM_COPY_HOST_PTR, sizeof(unsigned char) * size, luminance.data());
cl::Buffer buf_output(context, CL_MEM_WRITE_ONLY, sizeof(unsigned char) * size);
cl::Buffer buf_hist(context, CL_MEM_READ_WRITE, sizeof(cl_uint) * num_bins);
cl::Buffer buf_scan(context, CL_MEM_READ_WRITE, sizeof(cl_uint) * num_bins);
cl::Buffer buf_lut(context, CL_MEM_READ_WRITE, sizeof(cl_uchar) * num_bins);
```

Figure 2: Buffer creation

```
const int num_bins = 256;
int size = width * height;
vector<unsigned char> output(size * spectrum);
```

Figure 3: Configurable bins

References

Cybulski, R. (2019) *Answer to "Filling and saving shared buffer between threads"*
Available from <https://stackoverflow.com/a/56651424> [accessed 4 April 2025]

Histogram Equalization - an overview | ScienceDirect Topics (n.d.) Available
from <https://www.sciencedirect.com/topics/computer-science/histogram-equalization#> [accessed 4 April 2025]

Juju (2019) *Filling and saving shared buffer between threads* Available from
<https://stackoverflow.com/q/56651127> [accessed 4 April 2025]

Rodrigues, B. (2020) *What is OpenCL* Available from <https://medium.com/@bcrodrigues/what-is-opengl-14fbec353e09> [accessed 4 April 2025]

Sengupta, S., Lefohn, A. E. and Owens, J. D. (n.d.) *A Work-Efficient Step-Efficient Prefix-Sum Algorithm*.

The OpenCL™ Specification (n.d.) Available from https://registry.khronos.org/OpenCL/specs/3.0-unified/html/OpenCL_API.html [accessed 4 April 2025]

Tschumperle, D., Tilmant, C. and Barra, V. (2023) *Digital Image Processing with C++: Implementing Reference Algorithms with the CImg Library*. Boca Raton: CRC Press

What is a LUT? Ultimate Guide to Color Grading [FREE LUT Pack] (2025)
Available from <https://www.studiobinder.com/blog/what-is-lut/> [accessed 4 April 2025]